

WORLDLINE GLOBAL — ROUND 2

SDET / Test Automation Lead

Topic-wise Interview Preparation • Part 1

Java Defensive Coding • Exceptions • Concurrency • Test Design • API Idempotency & PATCH

How to use this PDF

This is based on our actual Worldline Round 2 practice. Each topic focuses on **simple explanation, practical example, interview-ready answer, and common interview trap**. We will keep future topics in separate topic-wise PDFs.

1. Defensive Coding & Money Calculation

Code reviewed:

```
public class OrderService {
    private double total = 0;

    public void addItem(Double price, Integer quantity) {
        if (quantity > 0) {
            total = total + price * quantity;
        }
    }

    public double getTotal() {
        return total;
    }
}
```

Problems:

- **Null:** Double and Integer are nullable wrappers. Arithmetic auto-unboxes them; null can cause NullPointerException.
- **Quantity:** zero and negative values should be rejected if invalid by business rules.
- **Price:** negative price should normally be rejected; price zero can be valid for free/promotional items.
- **Money:** double can have floating-point precision issues. Use BigDecimal.
- **Concurrency:** total is shared mutable state; concurrent read-modify-write can lose updates.

Interview-ready

I would validate null and invalid values at the boundary, reject invalid quantity/price according to business rules, use BigDecimal for money instead of double, and make the shared total thread-safe if the service can be accessed concurrently.

2. Concurrency & Thread Safety

The line **total = total + price * quantity** is conceptually read → calculate → write. Two threads can interleave these steps.

Race condition

If total=100, Thread 1 adds 50 and Thread 2 adds 30. Both can read 100. One writes 150, the other writes 130. Expected=180, actual=130. One update is lost.

synchronized

```
public synchronized void addItem(Double price, Integer quantity) {  
    // validate input  
    // update total  
}
```

For the same object instance, synchronized allows one thread at a time into the synchronized method.

Interview-ready

total is shared mutable state. The read-modify-write operation is not atomic, so concurrent threads can cause a race condition and lost updates. synchronized can protect the critical section within the same JVM/lock.

AtomicInteger

```
private AtomicInteger count = new AtomicInteger(0);  
  
public void increment() {  
    count.incrementAndGet();  
}  
  
public int getCount() {  
    return count.get();  
}
```

AtomicInteger suits simple shared integer counters. It is not automatically better than synchronized when several related values or a larger critical section must be protected.

ThreadLocal

```
ThreadLocal<WebDriver> driver;
```

ThreadLocal gives each thread its own isolated value. It does not solve a shared counter. In parallel UI automation, it can keep a separate WebDriver per test thread.

Easy comparison

AtomicInteger: shared numeric state + atomic operations.

synchronized: protects a critical section.

ThreadLocal: separate state per thread.

3. Exception Handling

Java hierarchy at interview level:

```
Throwable
├── Exception
│   ├── RuntimeException
│   │   ├── NullPointerException
│   │   ├── IllegalArgumentException
│   │   ├── ArithmeticException
│   │   └── Checked exceptions
│   └── IOException
└── Error
    ├── OutOfMemoryError
    └── StackOverflowError
```

Exception: application/runtime condition that can often be handled or meaningfully propagated.

Error: serious JVM/system-level problem. Normally do not catch Error broadly.

Interview trap

A syntax/compile-time problem is not a Java Error object. The compiler reports compile-time errors before the program runs.

Checked vs unchecked

Checked: compiler forces handling or declaration, e.g. `IOException`.

Unchecked: extends `RuntimeException`; compiler does not force handling at compile time, e.g. `NPE`, `ArithmeticException`, `IllegalArgumentException`.

throw vs throws

```
throw new InvalidAmountException("Amount must be greater than zero");

public void withdraw(double amount)
    throws InvalidAmountException {
    // ...
}
```

throw actually raises an exception. **throws** declares possible propagation.

Custom checked exception

```
public class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

public void withdraw(double amount)
    throws InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException(
            "Amount must be greater than zero");
    }
}
```

3A. Exception Handling — Practical Patterns

finally and try-with-resources

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Cleanup/action");
}
```

finally normally executes whether an exception occurs or not. For AutoCloseable resources, modern Java commonly prefers try-with-resources.

```
try (FileReader reader = new FileReader("data.txt")) {
    // use reader
} catch (IOException e) {
    // handle/log
}
```

Try-with-resources automatically closes AutoCloseable resources, even when an exception occurs.

Catch the most specific exception you can meaningfully handle

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    // handle expected arithmetic problem
}
```

Avoid catch(Exception e) everywhere: it can hide unexpected defects.

Exception swallowing

```
try {
    processPayment();
} catch (Exception e) {
    // do nothing
}
```

This silently ignores failure and may let a test/application continue in an invalid state.

Log and rethrow

```
try {
    processPayment();
} catch (Exception e) {
    logger.error("Payment processing failed", e);
    throw e;
}
```

Logging gives diagnostics; rethrowing prevents the failure from being hidden.

throw e vs wrapping

```
throw e;

// Or add context and preserve the original cause:
throw new RuntimeException(
    "Unable to process payment", e);
```

Wrap when you need context or translation to an application-level exception, while preserving the cause.

Avoid a huge try-catch around an entire test; let the test framework capture unexpected failures.

4. Test Case Design — Money Transfer API

POST /transfer

```
{
  "fromAccount": "A100",
  "toAccount": "B200",
  "amount": 1000.00
}
```

Requirement: amount > 0 and <= available balance. Assume balance = ■1000.

Senior-level test design should be structured, not random.

Category	Examples
Positive	■1, ■100, ■500, ■999, ■1000
Negative	-■1, -■100, ■0, null, "abc", empty, missing
Lower boundary	-1 ■, 0 ■, 0.01 ■
Upper boundary	999.99 ■, 1000.00 ■, 1000.01 ■
Balance	Exact balance succeeds; just above balance fails
Concurrency	Two simultaneous ■1000 transfers; only one succeeds

Interview-ready

I would cover positive, negative, boundary, data validation, balance, and concurrency scenarios. I would test exactly 1000, just below it, and just above it, and verify that concurrent transfers cannot spend the same balance twice.

5. How to Automate Concurrent API Requests

Key distinction: the test creates concurrency; the application protects shared state.

Using synchronized in the test does not create simultaneous API calls.

Java conceptual example

```
ExecutorService executor =  
    Executors.newFixedThreadPool(2);  
  
Future<Response> response1 =  
    executor.submit(() -> transfer(1000));  
  
Future<Response> response2 =  
    executor.submit(() -> transfer(1000));  
  
Response r1 = response1.get();  
Response r2 = response2.get();
```

For precise simultaneous starts, CountDownLatch can coordinate worker threads. For larger concurrency, tools such as JMeter, k6, Gatling or Newman can be used.

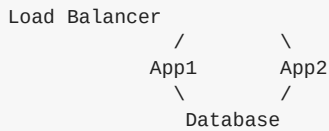
Verify:

- Exactly one request succeeds.
- The other fails with the expected insufficient-balance/business response.
- Final balance is correct.
- No duplicate transaction/double-spend occurs.

If you have not implemented it before

Do not bluff. Say: "I understand the approach conceptually. I would use ExecutorService/CountDownLatch for controlled concurrent requests, or a load tool for higher concurrency, then validate final state and transaction outcome."

6. Distributed Concurrency



If one transfer reaches App1 and another reaches App2, synchronized is not enough because each application instance has its own JVM/lock.

Interview-ready

synchronized only coordinates threads sharing the same lock within the same JVM. In a distributed deployment, App1 and App2 have separate locks, so concurrency control needs a shared layer such as database transactions, row-level locking, optimistic locking, an atomic database update, or an appropriate distributed lock.

Common approaches:

- Database transaction — make balance check and update one atomic unit.
- Row-level locking — lock the account row while checking/updating.
- Optimistic locking — use a version and detect conflicting updates.
- Atomic DB update — update only when sufficient balance exists.
- Distributed lock — possible when appropriate, but adds operational complexity.

7. API Idempotency — PUT, DELETE and POST

Definition

Idempotency means repeating the same request has the same intended effect on final server state as performing it once.

PUT — idempotent

```
PUT /users/101
{
  "name": "Swapnil",
  "city": "Pune"
}
```

Send it once or ten times: final resource state remains the same.

DELETE — idempotent

First DELETE may return 204 and a later DELETE may return 404, but the intended final state remains deleted. Idempotent does not mean identical responses.

POST — generally non-idempotent

```
POST /account/100/deposit
{
  "amount": 5000
}
```

Starting balance ■10,000. If processed three times as three deposits, final balance is ■25,000. Repeated requests create additional effects.

Payment retry problem

A payment can succeed while the client times out before receiving the response. A retry could charge twice unless the API supports idempotency.

```
POST /payment
Idempotency-Key: ABC123

{
  "amount": 5000
}
```

The server recognises the same key and returns the original result instead of creating another charge.

Method summary

Method	Typical	Key point
GET	Idempotent	Repeated reads do not normally change state
PUT	Idempotent	Same request leaves same intended final state
DELETE	Idempotent	Resource remains deleted
POST	Generally non-idempotent	Repeated request may create another effect
PATCH	Depends	Depends on operation semantics

Interview-ready

Idempotency means repeating the same request has the same intended effect on final state as one request. GET, PUT and DELETE are idempotent; POST is generally non-idempotent. For payments, idempotency keys prevent duplicate charges during retries.

8. PATCH — The Interview Trap

Question: Is PATCH idempotent or non-idempotent?

Correct answer

PATCH can be either idempotent or non-idempotent. It depends on the semantics of the operation.

Idempotent PATCH — set a fixed value

```
PATCH /users/101
{
  "city": "Mumbai"
}
```

Pune → Mumbai. Sending the same PATCH three times still leaves city=Mumbai. No additional state change occurs.

Non-idempotent PATCH — relative change

```
PATCH /account/100
{
  "operation": "incrementBalance",
  "amount": 100
}
```

■1000 → ■1100 → ■1200 → ■1300. Each retry changes the state again.

Easy memory rule

Set to a fixed value → can be idempotent.

Increment/add/relative change → non-idempotent.

Do not simply say “PATCH is non-idempotent.”

Best interview wording

PATCH is not inherently idempotent or non-idempotent. It depends on the operation. If the PATCH sets a field to a specific value, repeating it can be idempotent. If it performs a relative operation such as incrementing a balance, repeating it can produce a different final state, making it non-idempotent.

9. Last-Minute Revision Sheet

Money	BigDecimal, not double, for monetary precision.
Null	Validate nullable wrappers before arithmetic.
Validation	Reject invalid quantity/price according to business rules.
Race condition	Concurrent shared read-modify-write can lose updates.
synchronized	Protects shared state within the same JVM/lock.
AtomicInteger	Simple shared integer counters with atomic operations.
ThreadLocal	Separate state per thread; not a shared-counter solution.
Checked	Compiler forces handling or declaration.
Unchecked	Compiler does not force handling at compile time.
throw	Actually raises an exception.
throws	Declares possible propagation.
finally	Normally executes regardless of exception.
try-with-resources	Automatically closes AutoCloseable resources.
Exception swallowing	Never silently ignore important failures.
Test design	Positive + negative + boundary + validation + concurrency.
Concurrency test	ExecutorService/CountDownLatch or load tools create concurrent requests.
Distributed concurrency	Use shared-layer control such as DB transactions/locking.
PUT	Idempotent.
DELETE	Idempotent.
POST	Generally non-idempotent.
PATCH	Depends on operation semantics.
Payment retry	Use idempotency key to avoid duplicate charges.

Next in our chat

Question 30 — API Testing + Mocking: Your application calls a third-party Payment Service, but the Payment Gateway is down. How would you test payment functionality?